

HLD Chapter 0: Intuitive Distributed Systems Primer & Visual Metaphors

Core Learning Objective: Eliminate the abstract intimidation of High-Level Design. Master every core distributed systems building block using vivid real-world analogies, understand the 10 systems side-by-side in a single matrix, and practice with the 45-minute blank interview template.

1. The Distributed Systems Metaphor Dictionary (Zero Jargon)

Whenever a distributed systems concept feels confusing, ground your thinking in these intuitive physical analogies:

```
flowchart TD
```

```
    subgraph Analogies ["The Intuitive Distributed Systems Metaphor Dictionary"]
```

```
        LB["Load Balancer"] --- M_LB["Air Traffic Controller<br/>Directs incoming planes
```

```
        Cache["Redis Cache"] --- M_Cache["Bartender's Speed Rail<br/>Popular drinks kept
```

```
        WAL["Write-Ahead Log (WAL)"] --- M_WAL["Waiter's Order Notepad<br/>Order jotted d
```

```
        Cons_Hash["Consistent Hashing"] --- M_CH["Round Pizza Wheel<br/>Add a new slice;
```

```
        Sagas["Saga Orchestration"] --- M_Saga["Bank Wire Reversal<br/>If transfer fails
```

```
        WS["WebSockets"] --- M_WS["Open Telephone Call<br/>Direct, two-way open line vs n
```

```
    end
```

2. The 13 High-Level Systems Rosetta Stone Comparison

Click on any system to jump directly into its end-to-end architecture, back-of-the-envelope math, database schema, and failure mode deep dives:

System	Traffic Type	Read : Write Ratio	Primary Storage	Primary Cache	The Single Hardest Bottleneck
1. Messaging App (WhatsApp)	Bi-directional Real-Time	\$1 : 1\$	Cassandra (LSM-Tree)	Redis (Session registry)	50M open WebSockets + offline delivery
2. Ticketing Platform	Flash Spikes (Concerts)	\$5 : 1\$	PostgreSQL (ACID)	Redis (Distributed lock + TTL)	Zero double-booking across 50,000 users in 5s
3. Photo Feed (Instagram)	Read-Heavy (Photos)	\$50 : 1\$	Cassandra + S3	CDN (Edge) + Redis (Timelines)	Celebrity fan-out on write explosion
4. Distributed Scheduler	Compute / Batch Heavy	\$1 : 1\$	PostgreSQL + etcd	Timing Wheel in RAM	Worker crash failure recovery + leader election
5. Video Streaming (YouTube)	High Bandwidth Egress	\$100 : 1\$	S3 (Object Store)	Multi-Tier CDN Edge	Adaptive Bitrate chunking without playback stall
6. E-Commerce (Amazon)	Financial Transactions	\$10 : 1\$	MySQL (Partitioned)	Redis (Atomic Lua inventory)	Multi-service inventory/payment Saga rollback
7. Proximity Service (Yelp)	Geospatial Read-Heavy	\$100 : 1\$	PostGIS + ScyllaDB	Redis (H3 Spatial Indices)	Fast nearest-neighbor search without 2D lag
8. Mutual Match (Tinder)	High Write / Swipes	\$1 : 1\$	DynamoDB / Cassandra	Redis (Sets of user likes)	Sub-10ms mutual match detection on swipes
9. Ride Dispatch (Uber)	Massive GPS Ingestion	\$1 : 5\$ (Write Heavy!)	CockroachDB (Trips)	Redis (H3 Spatial driver grid)	1.25M GPS writes/sec without DB meltdown
10. Social Feed (Twitter/X)	Read-Heavy Microblogging	\$10 : 1\$	Cassandra + S3	Redis Cluster (800 IDs timeline)	Celebrity fan-out hybrid push-pull architecture
11. Distributed Cache (Redis)	Ultra Low-Latency In-Memory	\$10 : 1\$	Memory (RAM + RDB/AOF)	Redis Master-	Split-brain prevention &

System	Traffic Type	Read : Write Ratio	Primary Storage	Primary Cache	The Single Hardest Bottleneck
				Replica Shards	consistent hash slot rebalancing
12. Search Engine (Elasticsearch)	High-Volume Inverted Index	\$5 : 1\$	Lucene Segments (Disk)	File System Page Cache	Near real-time index refresh & segment merge throttling
13. Time-Series (Prometheus)	Append-Only Metrics Flush	\$1 : 100\$ (Write Heavy!)	Gorilla TSDB Chunking	Head Chunk Memory Buffer	10M metric samples/sec ingestion & compactor downsampling

3. The 45-Minute Printable Interview Cheat Template

Print this 1-page canvas to structure your whiteboard during mock interviews:

[00:00 - 05:00] PHASE 1: REQUIREMENTS

- [05:00 - 10:00] PHASE 2: MATH & ESTIMATIONS

- [10:00 - 25:00] PHASE 3: HIGH-LEVEL ARCHITECTURE

[25:00 - 38:00] PHASE 4: THE DEEP DIVE (Pick 2 Core Bottlenecks)

- [38:00 - 45:00] PHASE 5: RESILIENCE & FAILURE MODES

- Single Point of Failure (SPOF) eliminations?
- Cross-region replication strategy (Active-Active vs Active-Passive)?
- Disaster Recovery / Observability (Metrics, Traces, Alerts)?